Meal Inventory Management Project Report

Team: Mia Glenn, Maday Perez, Dev Patel, Lilly Galietta

1. Summary

Our project tasked us to design and implement a database inventory management system for the manufacturing of prepared meals. Our decision decisions were focused on supporting the required functionality while ensuring accurate outputs, a traceable inventory, and a normalized schema. Through the refinement of our design (from 'Deliverable 1' to 'Deliverable 2'), we improved normalization, resolved ambiguities, and supported data integrity in order to fully implement the required functionality.

2. ER Diagram Modifications

2.1 Summary of Design Changes

We designed an ER diagram to capture the entities and relationships necessary for modeling the requirements described in the project description. Our initial ER diagram (Appendix A), captured the core entities but included redundancies, lacked consistency, and did not properly support traceability and other design specifications. In 'Deliverable 2' we focused on solving these issues by improving normalization, resolving ambiguities, and considering all functional requirements and triggers in our ER Diagram (Appendix B).

2.2 Modified Subtypes

The first major improvement we made to the ER Diagram, is refactoring the design to account for the 'One Role' requirement. In version 1 (V1), Supplier, Viewer, and Manufacturer were subtypes (IsA) of the User table. However, this relationship is incorrect as it implies that they are separate entities from User. Instead, in version 2 (V2) we removed the subtype tables from User, and used the user_type attribute and a separate Supplier entity to specify the type of user. The supplier entity is able to reference a user if an account exists. Representing roles in this manner enforces the 'User's can have one role' requirement cleanly. Additionally, by having a separate entity for Supplier it supports the representation of external suppliers.

2.3 Altered Key Usage for Relationship Tables

The second improvement made in V2 involved altering the primary key composition for several many-to-many relationships. Initially, tables like SupplierIngredient and FormulationMaterial, had primary keys that were formed by artificial IDs which limited uniqueness and normalization capabilities. In V2, we altered the design so that the foreign keys now form the primary key which also helps with preventing accidental duplicate rows.

2.4 Naming Clarity

Finally, in V2 we analyzed each of the entity's names and attribute names. When looking at these names we considered what the entity or attribute is meant to represent and achieve in the database, and then determined if the name suitably communicated this functionality. If a name was unclear, it was adjusted to properly communicate its functionality (e.g, DoNotCombine → ConflictPair, BatchIngredientUsage → ProductionConsumption, RecipePlanDetail → ProductBOM).

3. Functional Dependencies & Design Impacts

Functional dependencies guided our design decisions and changes in our ER Model, and led to relational decompositions. Identifying functional dependencies allowed us to ensure that the relations avoided redundancy, and partial and transitive dependencies. The FDs had a large influence on our decomposition decisions around associative relationships, recipe versioning, conflict rules, and supplier formulations. Appendix C displays the complete list of functional dependencies used in our design. This portion of the report highlights the FDs that had major impacts on our design decisions.

An example of how FDs drove decomposition is the use of composite foreign keys in associative tables. In V1 of the ER diagram many tables used artificial primary keys. This is evident in the FormulationMaterial table. In V1 of the ER diagram, FormulationMaterial had 'formulation_material_id' as its key. However, upon looking at the FDs we determined that each of the non-key attributes was being functionally determined by a set of foreign keys for many tables. The FD in the case of FormulationMaterial shows that (formulation_id, ingredient_id) → qty. So, formulation_id and ingredient_id determine qty. Thus, by looking at these FDs we noticed that the use of artificial keys was introducing partial dependencies. This observation led to decomposition in V2 of the ER diagram as we redefined the keys in these tables so that their foreign keys compose the primary keys rather than the use of artificial keys.

Additionally, the batch entities (IngredientBatch and ProductBatch) are structured based off of FDs. The FD that lot_number fully determines all of the batch attributes is what guides the design of the batch entities and ensures that the behavior of the batches is stable. Another example of an FD that impacted our design decisions is (product_id, ingredient_id, recipe_id) → qty seen in the ProductBOM table. This FD revealed that a composite key was necessary in order to avoid ambiguity in ingredient usage across the different recipe versions. This FD led to decomposition in which each BOM entry is now uniquely tied to a specific product and recipe version. Overall, functional dependencies drove our design decisions and helped to reduce redundancy, clarify relationships, and prevent anomalies. All the functional dependencies which impact our decision making are available in Appendix C.

4. Normalization Analysis

Normalization played a central role in refinement between Deliverable 1 and Deliverable 2. After identifying the functional dependencies (Appendix C), we ensured that all of the tables were in at least Third Normal Form (3NF), with most reaching Boyce Codd Normal Form (BCNF). Each table eliminates partial and transitive dependencies, ensuring data consistency and minimizing redundancy. A breakdown of the normalization of the tables is available in Appendix D.

4.1 3NF

3NF tables intentionally preserve design flexibility for self-references or optional relationships. RecipePlan and RecallEvent are in 3NF, while the other tables are in BCNF. RecipePlan is in 3NF because recipe_id determines all attributes. The self-referencing attribute replaced_by_id is used for version control and does not introduce transitive dependencies. RecallEvent is in 3NF because recall_id determines all attributes. Some fields, such as ingredient_id and ingredient_lot, are nullable and depend only on recall context, but this does not violate 3NF.

4.2 BCNF

The remaining tables are in BCNF, BCNF tables ensure elimination of anomalies in insert, update, or delete operations. User is in BCNF because user_id is the primary key, and all other attributes depend only on it, with no transitive dependencies. Category is in BCNF because category_id uniquely determines the name, which is atomic and unique. Product is in BCNF because product_id uniquely determines all non-key attributes. The foreign keys category_id and manufacturer_id do not introduce transitive dependencies. Ingredient is in BCNF because ingredient_id determines all other attributes, with no partial or transitive dependencies. Supplier is in BCNF because supplier_id uniquely determines all other attributes, and the relationship to the User table is represented through a foreign key only. SupplierIngredient is in BCNF because its composite primary key (supplier_id, ingredient_id) determines can_supply, and all non-key attributes are fully dependent on the entire key. IngredientFormulation is in BCNF because the composite key (ingredient_id, supplier_id, version_no) determines all attributes. No attribute depends on part of the key. FormulationMaterials is in BCNF because the composite key (formulation_id, ingredient_id) determines qty, with no partial dependencies. ProductBOM is in BCNF because the composite key (product_id, ingredient_id, recipe_id) determines qty, and all attributes depend on the entire key. IngredientBatch is in BCNF because lot_number determines all other attributes. All non-key attributes depend fully on the primary key. ProductBatch is in BCNF because lot_number determines all other attributes with no transitive dependencies. ProductionConsumption is in BCNF because the composite primary key (product_batch, ingredient_lot) determines qty, which depends entirely on the full key. ConflictPairs is in BCNF because the composite key (ingredient_a, ingredient_b) determines reason and severity. The symmetric pair constraint is enforced by a trigger, and no partial dependencies exist. HealthRiskRule is in BCNF because rule_id determines all other attributes, with no partial or transitive dependencies. IngredientHealthRisk is in BCNF because the composite uniqueness ensures that each pair determines threshold_quantity, with no redundant dependencies.

5. Constraints

In the database, triggers and procedures implemented constraints that required computation or coordination across several tables. Constraints that are too complex or need too much context to handle at the database level are moved into the application code.

5.1 Constraints Implemented in the Database

The lot number format constraint ensures that IngredientBatch lot_number's follow a specific pattern for combining ingredient_id, supplier_id, and batch_id. This requires automatically generating a value by concatenating multiple fields which cannot be done with a simple CHECK. Instead, we used a BEFORE INSERT trigger on the IngredientBatch table to compute and validate the lot number format before records are inserted.

The constraint which prevents the consumption of expired ingredients during production involves the comparison of the current date and the expiration date in the IngredientBatch table which cannot be done with a CHECK since it involves multi-table coordination. Instead, we used a BEFORE INSERT trigger on the ProductionConsumption table to check the dates of any ingredient lot being consumed and reject if expired ingredients are detected.

Maintaining accurate on-hand quantities involves updating the IngredientBatch table when records are inserted into the ProductionConsumption table. This could not be handled by a simple constraint, so we used an AFTER INSERT trigger on ProductionConsumption to automatically decrement on_hand_quantity in IngredientBatch by the amount consumed.

Another constraint involves active formulations. A supplier cannot have multiple active formulations for the same ingredient with overlapping effective dates. A CHECK constraint cannot achieve this as it requires comparing date ranges of new records against all existing records for that ingredient-supplier combination. Instead, we used a BEFORE INSERT or UPDATE trigger on IngredientFormulation that validates the dates do not overlap.

5.2 Constraints Implemented in the Application Code

The standard batch size constraint states that production quantities have to be integer multiples of the standard batch size. Enforcing this as a trigger would be messy, instead validate this in the application since it is simpler and lets us show clear error messages instead of database errors.

Implementing the FEFO (First Expired, First Out) logic in SQL triggers is complex as it requires sorting lots by expiration and pulling quantities from multiple batches if needed. In the application, we can simply query the lots, sort them, and loop until the required quantity is met.

The one-level composition constraint involves looking at each material in a compound ingredient to verify it is atomic. This is possible through recursive SQL queries but that is complex and application code results in a more clear and maintainable implementation.

The do-not-combine warnings require us to expand out compound ingredients and check all pairs in the ConflictPairs table when a recipe or production batch is created. This is too complex to cleanly implement in the database, but can be done cleanly in the application by expanding ingredients, generating all possible pairs, and checking for conflict through the database.

Access control is difficult in the database as it is unaware of who is logged in. The application is capable of tracking the current user and their permissions to appropriately limit their available operations.

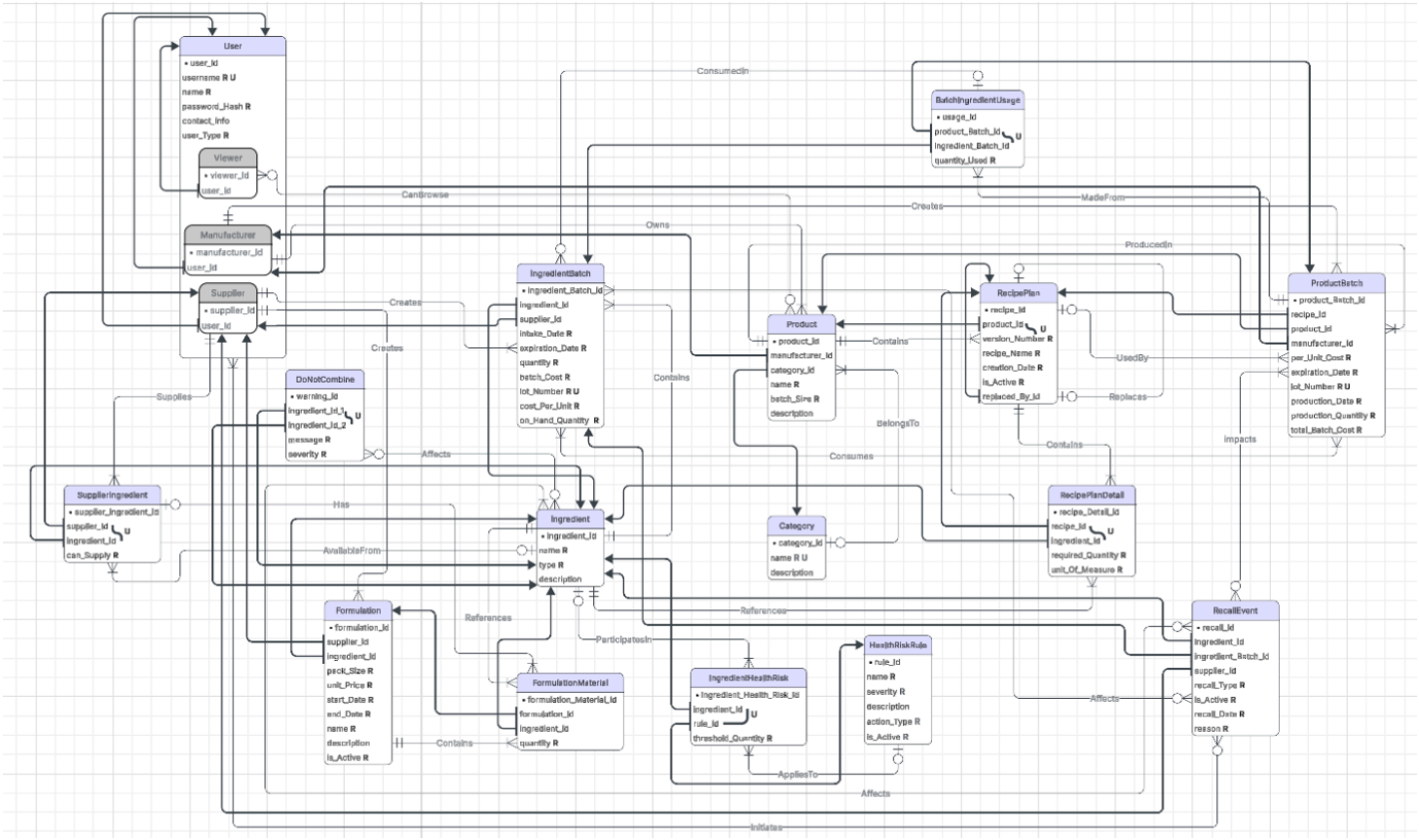
Checking ingredient quantity availability involves performing a summation of quantities from multiple ingredient lots and comparing them to the recipe requirements which is simple and efficient to be performed in the application, and allows users to quickly alter their choices.

Recall traceability constraint requires all product batches that use certain ingredients to be found within a time window. We have a stored procedure that handles table joins and finding connections, but the application is better suited for filtering and formatting results.

Appendix A

Entity-Relationship (ER) Diagram – Version 1

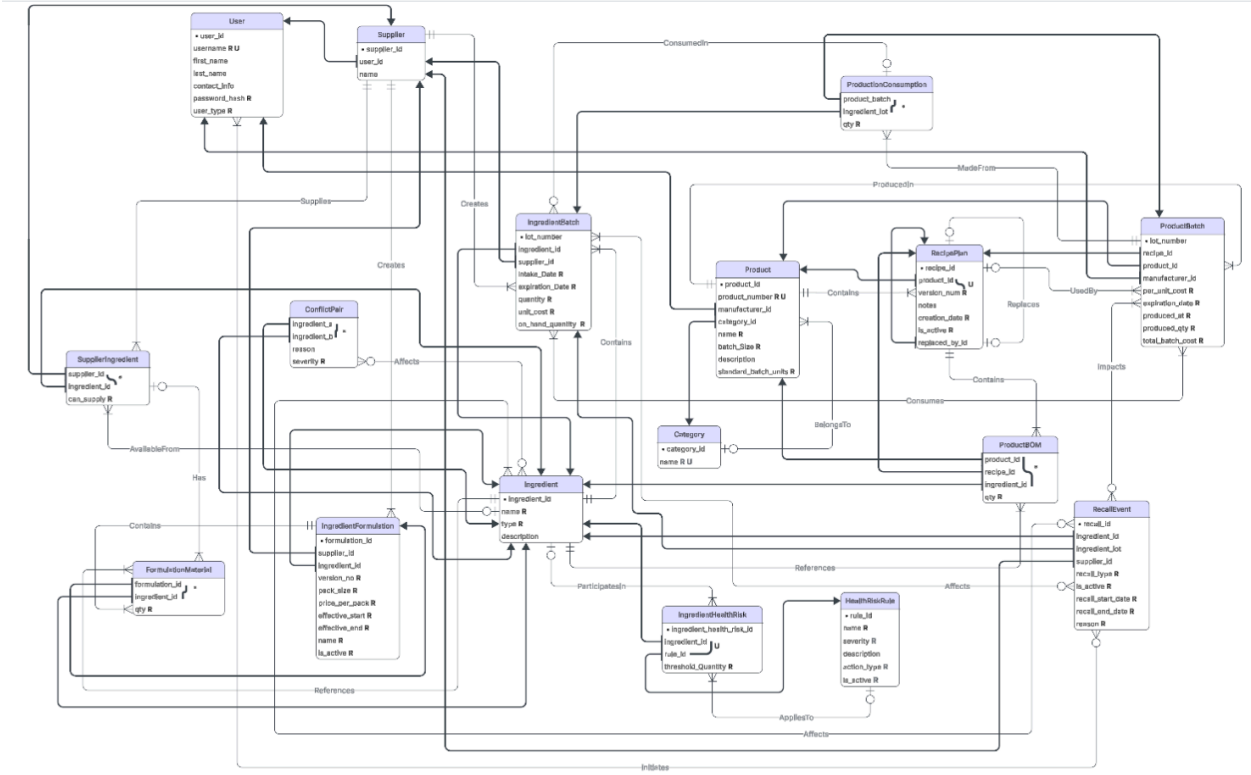
This is our ER diagram submitted for Deliverable 1



Appendix B

Entity-Relationship (ER) Diagram – Version 2

This is our ER diagram submitted for Deliverable 2



Appendix C

Functional Dependencies Table

Table	Key	Functional Dependencies
User	user_id	user_id → username, first_name, last_name, contact_info, password_hash, user_type
Supplier	supplier_id	supplier_id → user_id, name
SupplierIngrdient	(supplier_id, ingredient_id)	(supplier_id, ingredient_id) → can_supply
ConflictPair	(ingredient_a, ingredient_b)	(ingredient_a, ingredient_b) → reason, severity
FomulationMaterial	(formulation_id, ingredient_id)	(formulation_id, ingredient_id) → qty
IngredientFormulation	formulation_id	supplier_id, ingredient_id, version_no, pack_size, price_per_pack, effective_start, effective_end, name, is_active
Ingredient	ingredient_id	ingredient_id → name, type, description
IngredientBatch	lot_number	lot_number → ingredient_id, supplier_id, intake_date, quantity, unit_cost, on_hand_quantity
Category	category_id	category_id → name
IngredientHealthRisk	ingredient_health_risk	ingredient_health_risk → ingredient_id, rule_id, threshold_quantity
HealthRiskRule	rule_id	rule_id → name, severity, description, action_type, is_active
Product	product_id	product_id → product_number, manufacturer_id, category_id, name, batch_size, description, standard_batch_units
ProductConsumption	(product_batch, ingredient_lot)	(product_batch, ingredient_lot) → qty
RecipePlan	recipe_id	recipe_id → product_id, version_num, notes, creation_date, is_active, replaced_by_id
ProductBOM	(product_id, recipe_id, ingredient_id)	(product_id, recipe_id, ingredient_id) → qty
RecallEvent	recall_id	recall_id → ingredient_id, ingredient_lot, supplier_id, recall_type, is_active, recall_start_date, recall_end_date, reason
ProductBatch	lot_number	lot_number → recipe_id, product_id, manufacturer_id, per_unit_cost, expiration_date, produced_at, produced_qty, total_batch_cost

Appendix D

Normalization Table

BCNF	**3NF**
User Supplier SupplierIngredient ConflictPair FormulationMaterial IngredientFormulation Ingredient IngredientBatch Category IngredienHealthRisk HealthRiskRule Product ProductConsumption ProductBOM ProductBatch	RecipePlan RecallEvent

Appendix E

Python CLI Flow

```
inventoryManagement % python3 main.py
============================================================
 CSC540 Project 1 - Inventory Management System
============================================================
Login
Select role:
[1] Manufacturer
[2] Supplier
[3] General (Viewer)
[4] View Queries
[0] Exit

Enter choice: 1

=== Manufacturer Login ===
Username: john_smith
Password: hash

============================================================
 [Manufacturer] - MFG001
============================================================
1) Define/Update Product
2) Define/Update Product BOM
3) Record Ingredient Receipt
4) Create Product Batch
5) Reports
6) (Grad) Recall/Traceability
0) Logout

=== Supplier Login ===
Username: jane_doe
Password: hash

============================================================
 [Supplier] - SUP001
============================================================
1) Declare Ingredients Supplied
2) Maintain Formulations
3) Create Ingredient Batch
4) (Grad) Maintain Do-Not-Combine List
0) Logout
```

```
=== Viewer Login ===
Username: bob_johnson
Password: hash

============================================================
 [General (Viewer)] - VIEW001
============================================================
1) Browse Product Types
2) Generate Ingredient List
3) (Grad) Compare Two Products for Incompatibilities
0) Logout

============================================================
 Predefined Queries
============================================================
1) List the ingredients and the lot number of the last batch of product
type Steak Dinner (100) made by manufacturer MFG001.
2) For manufacturer MFG002, list all the suppliers that they have purchased
from and the total amount of money they have spent with that supplier.
3) For product with lot number 100-MFG001-B0901, find the unit cost for
that product.
4) Based on the ingredients currently in product lot number
100-MFG001-B0901, what are all ingredients that cannot be included (i.e.
that are in conflict with the current ingredient list).
5) Which manufacturers have supplier Supplier B (21) NOT supplied to?
0) Back
```